

Week 5 - The Advanced Encryption Standard (AES) and RSA Algorithm

MAT260: Cryptology

Gary Hobson

Date June 2, 2025

Introduction:

The assigned computer problems this week are from Chapter 9, Problems 1, 2, 3, and 6.

Make sure to review the example computer problems in "Appendix C.6 Examples for Chapter 9" that work similar problems to those you are assigned.

Make sure to run your code so all relevant computations/results are displayed and then export your work as a PDF file for submission.

Chapter 9 Problems:

Problem 1:

To determine the plaintext without factoring n , I assumed the message was either "one" or "two" and encoded each as an integer using base-26. For the word "one":

$$15 \cdot 676 + 14 \cdot 26 + 5 = 10509$$

For the word "two":

$$20 \cdot 676 + 23 \cdot 26 + 15 = 14133$$

(resulting in 151405 and 202315, respectively). I then encrypted both using the RSA formula:

$$c = me \bmod n.$$

The ciphertext generated from "one" matched the received ciphertext (273095689186), confirming that "one" was the original plaintext.

To prevent this kind of guesswork, Paul Revere's friend could have added

random padding to the message before encryption (as described at the end of Section 9.2.2). This would make identical messages encrypt to different ciphertexts and protect against brute force guessing.

```
% RSA parameters
n = sym(712446816787);
e = sym(6551);
c = sym(273095689186);

% Possible plaintexts (adjusted based on user's comment)
m_one = sym(151405); % "one"
m_two = sym(202315); % "two"

% Compute ciphertexts
c_one = powermod(m_one, e, n);
c_two = powermod(m_two, e, n);

% Display results
disp(['Ciphertext for "one": ' num2str(double(c_one))]);
```

Ciphertext for "one": 273095689186

```
disp(['Ciphertext for "two": ' num2str(double(c_two))]);
```

Ciphertext for "two": 709427776011

```
disp(['Received ciphertext: ' num2str(double(c))]);
```

Received ciphertext: 273095689186

```
% Determine plaintext
if c_one == c
    disp('The plaintext is "one"');
elseif c_two == c
    disp('The plaintext is "two"');
else
    disp('Neither matches');
end
```

The plaintext is "one"

Problem 2:

To factor the RSA modulus $n = 718548065973745507$, I applied the $ar \equiv 1 \pmod{\phi(n)}$ method in Subsection 9.4.2. Given the public exponent $e = 3449$ and the private key $d = 543546506135745129$, I first computed:

Step 1. Compute $r = ed - 1$: $r = ed - 1 = 25 \cdot 58584121864443279685$

Step 2. Express r as $2^k \cdot m$ with m odd: I identified $k = 5$.

Step 3. Test values of a from 2 to 100: For each a , I computed $b = a^m \bmod n$ and checked successive squares of b

modulo n . If an intermediate square gave 1 but was not $n-1$, I computed $\gcd(b-1, n)$ to recover a nontrivial factor.

Using this process, I found that n factors as:

$p = 740876531$

$q = 969862097$

These values satisfy $p \cdot q = n$, confirming the result.

```
format long

% Use symbolic variables to avoid precision issues
n = sym('718548065973745507');
e = sym('3449');
d = sym('543546506135745129');

% Step 1: Compute r = ed - 1
r = e * d - 1;

% Step 2: Write r = 2^k * m with m odd
k = 0;
m = r;
while mod(m, 2) == 0
    m = m / 2;
    k = k + 1;
end

fprintf('r = 2^%d * %s\n', k, char(m));
```

```
r = 2^5 * 58584121864443279685
```

```
% Step 3: Try several random a until successful
success = false;
for a = 2:100
    if gcd(sym(a), n) ~= 1
        continue
    end

    b = powermod(sym(a), m, n);

    if b == 1 || b == n - 1
        continue
    end
```

Factors of n are:
 $p = 740876531$
 $q = 969862097$

Problem 3: - You cannot tell that 'cat', 'bat', and 'hat' differ by just one letter.

You *can* observe that 'encyclopedia' and 'antidisestablishmentarianism' are much longer,

as their ciphertexts result from much larger plaintext numbers.

4

```
e = sym('65537'); % Commonly used public exponent

% Step 3: Define plaintext messages
messages = {'cat', 'bat', 'hat', 'encyclopedia', 'antidisestablishmentarianism'};

% Step 4: Encode and encrypt each message
fprintf('Message\t\tEncoded m\t\tCiphertext\n');
```

Message	Encoded m	Ciphertext
---------	-----------	------------

```
%fprintf('-----\n');
for i = 1:length(messages)
    m = text2int_big(messages{i}); % Convert to integer
    c = powermod(m, e, n);          % Encrypt using RSA
    fprintf('%s\t\t%s\t\t%s\n', messages{i}, char(m), char(c));
end
```

cat	1371	1218184403261551994745217199095854908353539892691075023479
bat	695	4979844475190903314426154573798629948482530124692252866309
hat	4751	6979803540725646472871794170637517139891137965303607086689
encyclopedia	16532440683313100	2283309049418038110274728940252228557505995908575948626307
antidisestablishmentarianism	84602278762307761469177551207947970504	2873402376913204263858025785137103

Problem 6:

To factor $n = 537069139875071$, I used the fact that:

$a = 859753244431662 \equiv b = 4624361062612 \pmod{n}$,

which implies:

$a^2 \equiv b^2 \pmod{n}$.

This gives:

$(a - b)(a + b) \equiv 0 \pmod{n}$,

so $\gcd(a - b, n)$ should reveal a nontrivial factor of n . Using MATLAB, I computed the difference $a - b$, then calculated $\gcd(a - b, n)$ to find one factor.

Dividing n by this factor gave the second. The result was two nontrivial factors of n , confirming a correct factorization.

```
format long g

% Given values
n = sym('537069139875071');
a = sym('859753244431662');
b = sym('4624361062612');

% Step 1: Compute difference
```

```
diff = a - b;  
  
% Step 2: Compute GCD  
factor1 = gcd(diff, n);  
  
% Step 3: Get second factor  
factor2 = n / factor1;  
  
% Display results  
fprintf('Factor 1: %s\n', char(factor1));
```

Factor 1: 9876469

```
fprintf('Factor 2: %s\n', char(factor2));
```

Factor 2: 54378659